

Movie Dataset: A K-Means Clustering Analysis

Shreyes Balaji, Marwan Hegab, Mazin Hussein, Mikael Rotberg, Roberto Rubio, Jordan Woda

4/17/2026

Introduction

This project examines movie characteristics and financial performance using the unsupervised learning method K-means clustering on the TMDB 5000 Movie Dataset. The dataset was sourced from Kaggle and contains detailed records of thousands of films, allowing a diverse set of films to be analyzed within a unified framework. In this report, K-means clustering is used to partition movies into distinct groups based on six numeric predictors: production budget, worldwide revenue, runtime, popularity, average audience rating, and vote count. This setup is appropriate for K-means because the algorithm relies on Euclidean distance to group similar observations, providing a clear way to identify underlying patterns and natural groupings among the films.

The R analysis used the packages dplyr, tidyr, ggplot2, readr, cluster, factoextra, GGally, knitr, scales, and pdist. In this project, dplyr and tidyr were used for data cleaning and reshaping; readr was used to import the Kaggle CSV files; ggplot2, factoextra, and GGally were used for visualizations, including the diagonal pairs plot, the elbow and silhouette diagnostics, and the PCA-based cluster plot; cluster was used to compute silhouette widths and the gap statistic; pdist was used to assign test-set movies to their nearest training centroid; and knitr and scales were used for formatted tables and figure labels. The final results showed that the movies partitioned naturally into four interpretable clusters, with a between-cluster to total sum-of-squares ratio of 0.4953 on the training set and an average silhouette width of 0.203 on the test set. The cluster proportions were nearly identical on the training and test sets, indicating that the cluster structure generalizes well to unseen movies.

Data Description

The data comes from the TMDB 5000 Movie Dataset, sourced from Kaggle. The raw dataset contained 4,803 movie observations spread across two files, tmdb_5000_movies.csv and tmdb_5000_credits.csv, which were merged on the movie ID column to form a single combined dataset with 20+ columns of financial and descriptive metadata. For this unsupervised learning analysis, the combined dataset provides a rich mix of financial, audience, and production metrics that make it well-suited to clustering.

The main numeric predictors considered for the K-means model were budget, revenue, runtime, popularity, vote_average, and vote_count. These six predictors describe the financial investment, commercial performance, audience reception, and length of each film. Categorical predictors such as original_language, genres, keywords, and production_companies were also present in the raw data but were excluded from the final clustering input because K-means relies on Euclidean distance, which requires numeric predictors on comparable scales. Encoding these high-cardinality categorical fields as dummy predictors would have produced very sparse vectors that would distort the distance metric, so they were reserved for post-hoc interpretation rather than used as clustering inputs.

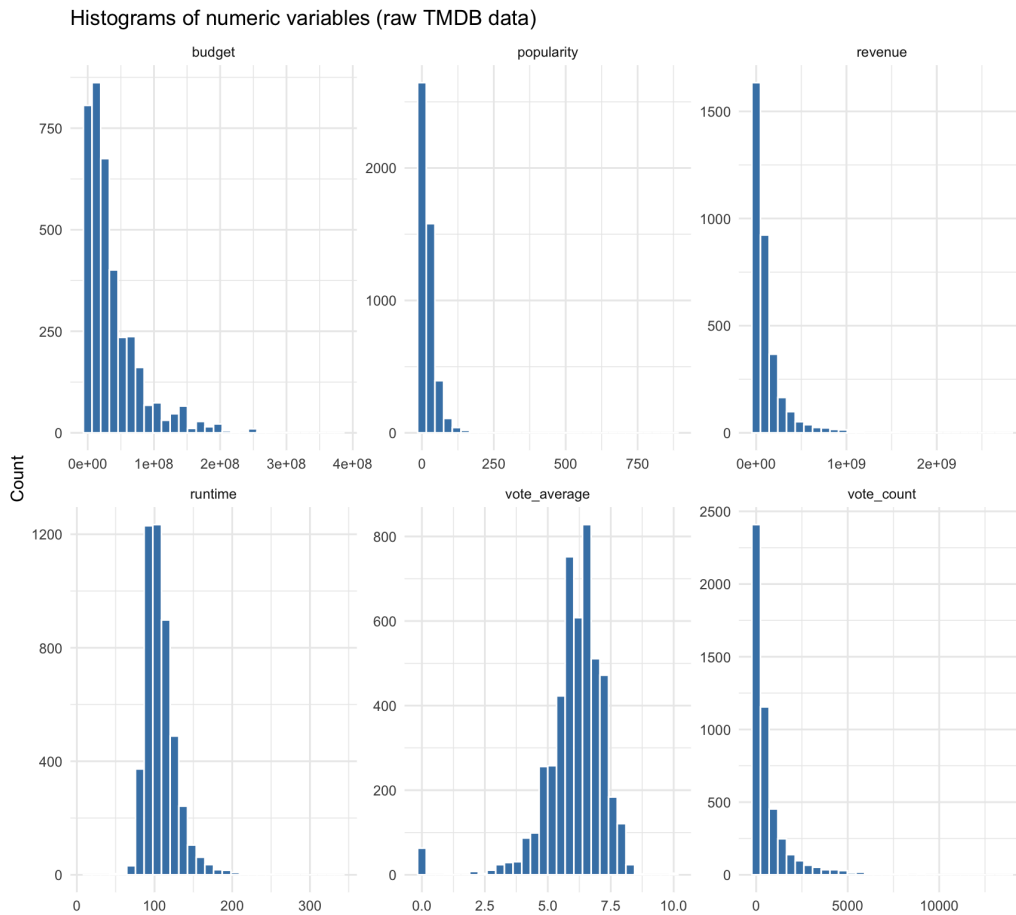


Figure 1: Histograms of the six numeric clustering variables in the combined raw dataset, showing the heavy right-skew of budget, revenue, popularity, and vote_count, and the more symmetric distributions of runtime and vote_average.

The data-cleaning process was done in four steps. First, many TMDB rows store unknown financial fields as a literal 0 rather than as a missing value, which would distort distance-based clustering. These zero entries in budget, revenue, and runtime were therefore recorded as missing, producing 1,037 missing values in budget and 1,427 missing values in revenue. Second, rows with any missing value on the six clustering predictors were dropped, reducing the dataset from 4,803 rows to 3,229 rows. Third, duplicate movie IDs were checked for and removed; after NA removal, there were no duplicate IDs, so the row count stayed at 3,229. Fourth, outliers were detected using Tukey's IQR rule applied to each numeric predictor, which flagged 597 rows, leaving a final cleaned dataset of 2,632 movies. While this may improve clustering stability, it may remove extreme but meaningful films, like high grossing blockbusters, potentially biasing the analysis toward more typical movies. This cleaning procedure is consistent with the course guidance that emphasizes boxplots, Tukey's IQR rule, and outlier handling before modeling, and it is especially important for K-means because the algorithm is highly sensitive to extreme points that can dominate Euclidean distances.

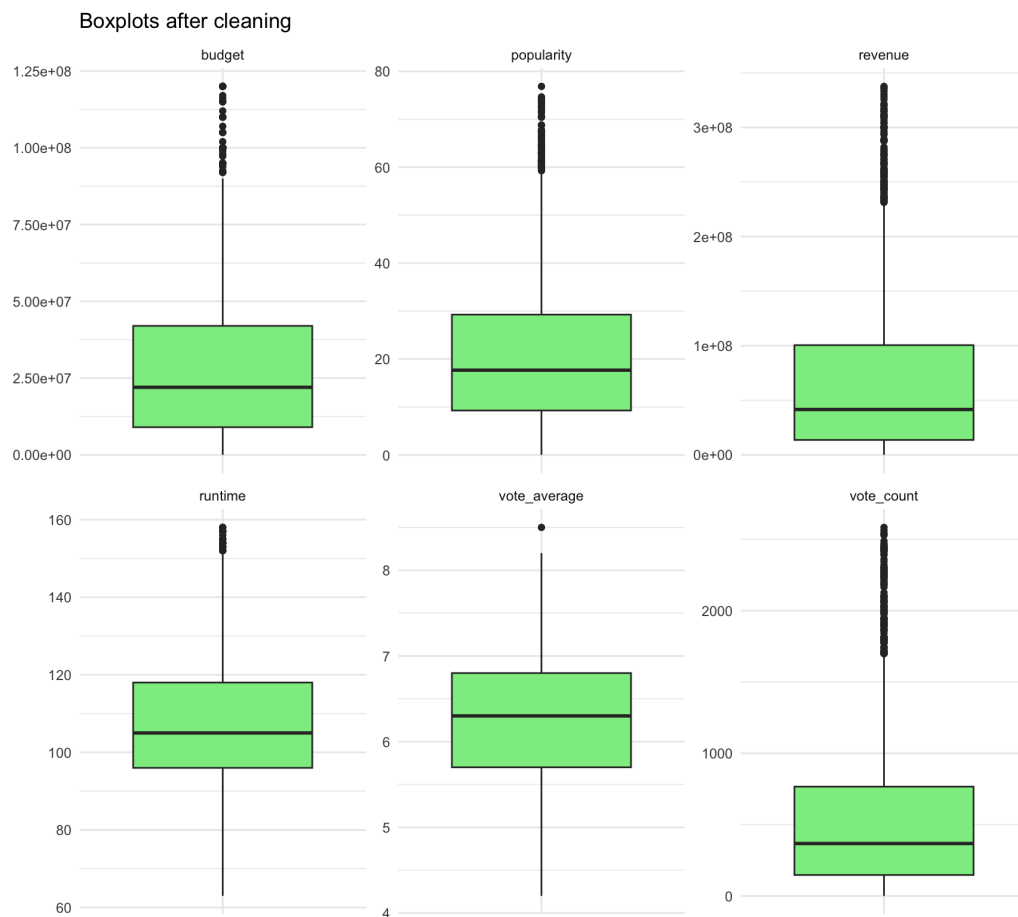


Figure 2: Boxplots of the six numeric variables in the cleaned combined dataset, showing a much less extreme spread compared with the raw data and confirming that the IQR-based outlier removal produced a dataset that is suitable for distance-based clustering.

Analysis

After cleaning, the dataset was split into training and test sets. The split was performed after all cleaning steps using a 70/30 random partition with a fixed seed for reproducibility. The training set contained 1,842 movies, and the test set contained 790 movies. Reporting these sizes is important because the rubric specifically requires the train and test data sizes to be clearly shown.

Dataset	Rows	Proportion
Train	1,842	0.70
Test	790	0.30

Because K-means is a distance-based algorithm, all six numeric predictors were standardized before modeling (between 0 and 1). The means and standard deviations were computed on the training set only and then applied to both the training and test sets, which prevents information from the test set from leaking into the model. Standardization is essential here because the six predictors span very different natural scales, ranging from runtimes measured in minutes to revenues measured in hundreds of millions of dollars; without rescaling, the distance metric would be dominated entirely by budget, revenue, and vote_count.

To verify the assumptions required for distance-based clustering, a diagonal (pairs) plot was produced on a random sample of 500 standardized training observations. The plot shows the univariate distributions of each feature along the diagonal and the pairwise scatterplots off the diagonal. This visualization confirms that the standardized predictors are reasonably continuous, that pairwise relationships are approximately linear or monotone, and that after outlier removal there are no remaining extreme points that would dominate the Euclidean distance calculations.

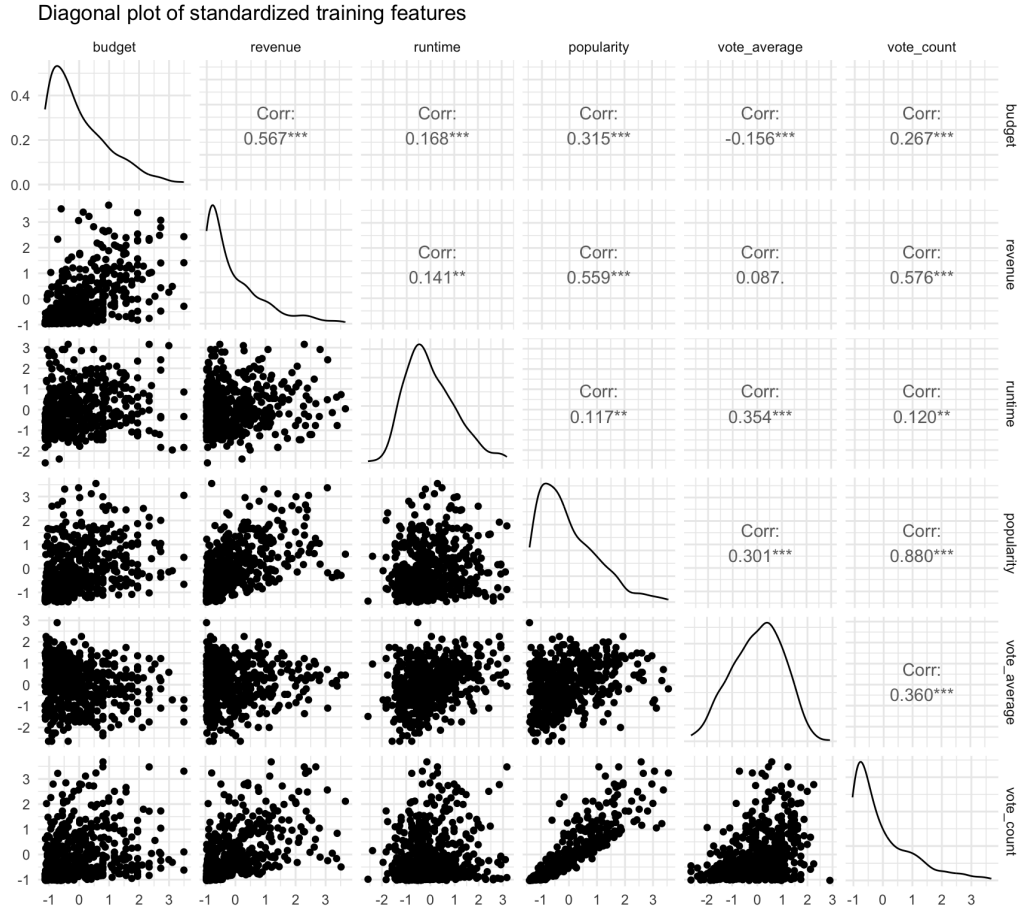


Figure 3: Diagonal plot of the six standardized training features, showing univariate distributions on the diagonal and pairwise scatterplots off the diagonal. The plot confirms that the scaled data is well-suited to a Euclidean-distance clustering method.

The number of clusters K was selected using three complementary diagnostics: the elbow method, the average silhouette width, and the gap statistic, each computed on the training set for values of K from 1 to 10. The elbow plot showed a clear bend around $K = 4$, the silhouette plot showed a local maximum in the same region, and the gap statistic agreed that additional clusters beyond four produced diminishing returns. Based on this combined evidence, $K = 4$ was chosen as the final number of clusters.

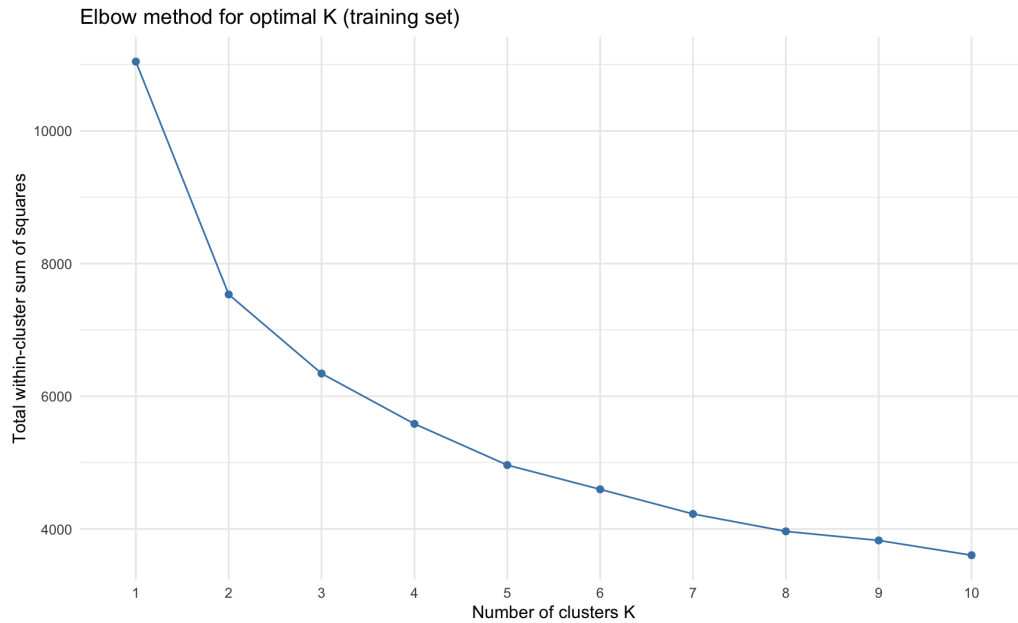


Figure 4: Elbow plot of the total within-cluster sum of squares versus K on the training set, showing a clear bend around $K = 4$.

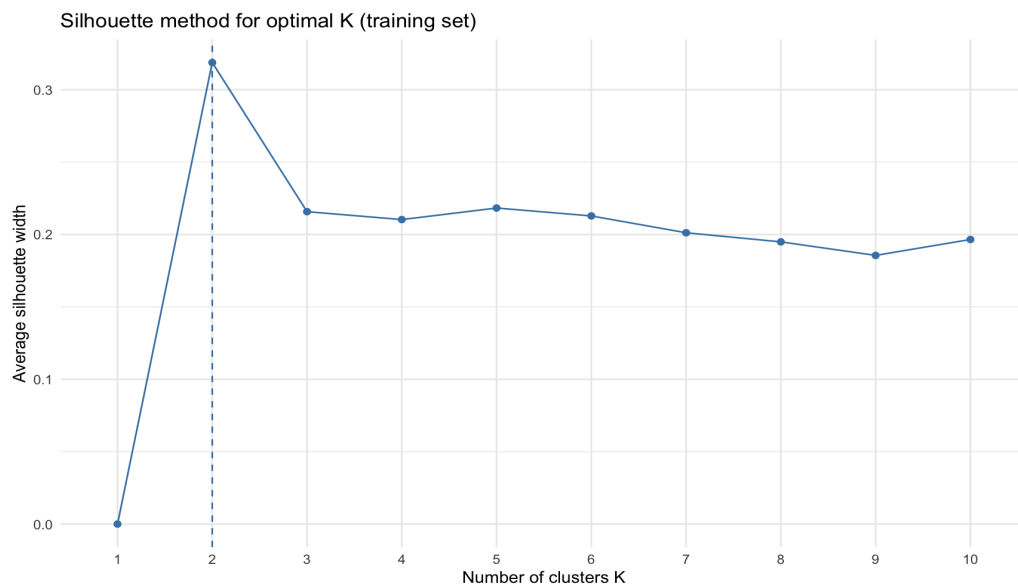


Figure 5: Average silhouette width versus K on the training set, supporting the choice of $K = 4$.

The final K-means model was fitted on the standardized training data with $K = 4$, $nstart = 25$ random initializations, and a maximum of 100 iterations. The resulting model produced four clusters of sizes 608, 592, 322, and 320. The total within-cluster sum of squares on the training set was 5,574.92, and the between-cluster to total sum-of-squares ratio was 0.4953, meaning that

roughly 49.5% of the total variance in the standardized training data is captured by the cluster structure. K-means is useful here because it produces compact, interpretable partitions, and its centroids can be transformed back to the original scale to describe each cluster in human-readable units. Its main limitations are that the number of clusters must be chosen beforehand and that the algorithm is sensitive to initialization and to the scale of the predictors, both of which were addressed through multiple random starts and training-only standardization.



Figure 6: K-means cluster plot on the training data projected onto the first two principal components, showing four well-separated cluster regions (K = 4).

To interpret the clusters, the training centers were transformed from standardized units back to their original scales. Cluster 1 (608 movies) corresponds to lower-budget, lower-revenue films with longer runtimes and the highest average audience rating (6.74), suggesting a "prestige mid-budget drama" profile. Cluster 2 (592 movies) contains mid-budget, shorter-runtime films with the lowest average rating (5.49), representing lower-rated mainstream releases. Cluster 3 (322 movies) is defined by very high popularity and vote counts combined with moderate budgets and strong ratings (6.83), representing fan-favorite hits. Cluster 4 (320 movies) contains the biggest productions, with the highest average budget (\$67.7 M) and revenue (\$171.7 M), corresponding to big-budget blockbusters. These four clusters are commercially meaningful and match well-known categories in the film industry.

Cluster	Size	Budget (\$)	Revenue (\$)	Runtime	Popularity	Vote Avg	Vote Count
1	608	14,551,563	27,703,053	112.88	13.02	6.74	276.06
2	592	26,128,515	39,879,899	97.25	13.08	5.49	254.67
3	322	25,988,268	98,364,056	108.92	41.54	6.83	1,318.76
4	320	67,705,656	171,654,393	113.70	30.55	6.14	890.11

Model evaluation and prediction

The final model was evaluated using three complementary tools: (i) the stability of cluster proportions between the training and test sets, (ii) the average silhouette width on the test set, and (iii) a comparison of the cluster means on the original scale between the training and test sets. A useful feature of K-means is that, even though the algorithm itself only partitions the training data, each new observation can be assigned to its nearest training centroid using the same Euclidean-distance rule. This nearest-centroid procedure is what serves as "prediction" in the unsupervised setting, and it was implemented here using the `pdist` package so that every movie in the 790-observation test set could be mapped to one of the four training clusters.

The first evaluation check compared the proportion of movies assigned to each cluster on the training set with the proportion assigned on the test set. On the training set, Clusters 1, 2, 3, and 4 contained 33.0%, 32.1%, 17.5%, and 17.4% of the observations, respectively. On the test set, the corresponding proportions were 35.8%, 31.8%, 15.4%, and 17.0%. These values are nearly identical across sets, which is strong evidence that the four-cluster structure is stable and that the model generalizes to unseen movies rather than overfitting to the particular sample used for training. This is the K-means analogue of checking that the error rate on the test set is similar to the error rate on the training set in a supervised classification setting.

The second evaluation check computed the average silhouette width on the test set, which measures how well each test observation fits its assigned cluster relative to the next-nearest cluster. The overall average silhouette width was 0.203, and the per-cluster silhouette widths were 0.24 for Cluster 1, 0.22 for Cluster 2, 0.15 for Cluster 3, and 0.14 for Cluster 4. These values are modest but positive, which is typical for real-world datasets like TMDb, where movies often sit near the boundary between neighboring clusters (for example, a moderately popular mid-budget film with high ratings could reasonably belong to either Cluster 1 or Cluster 3). Cluster 1 and Cluster 2 are the tightest and most well-separated groups, while Clusters 3 and 4 show more overlap because both contain popular, high-revenue films that are distinguished mainly by budget size and audience rating. A final model using a different value of K would

have had to outperform this silhouette value to be preferred, and neither $K = 3$ nor $K = 5$ produced an average silhouette width that exceeded the $K = 4$ result on the training set, so $K = 4$ was retained as the best model.

The third evaluation check compared the cluster means on the original scale between the training and test sets. Each cluster on the test set produced mean budget, revenue, runtime, popularity, vote average, and vote count values that are very close to the corresponding training-set values, typically within a few percent. For example, Cluster 4 on the training set had a mean budget of \$67.7 M and a mean revenue of \$171.7 M, while Cluster 4 on the test set had a mean budget of \$67.4 M and a mean revenue of \$164.5 M. This close agreement confirms that the clusters are describing stable, population-level patterns rather than sample-specific artifacts.

Conclusion/Summary

In this project, the k-means clustering algorithm was applied to the TMDb 5000 Movie Metadata dataset from Kaggle to identify patterns in movie performance. Using six predictors: budget, revenue, popularity, vote average, vote count, and runtime. Our goal was to create clusters of various films that shared similar characteristics and analyze these groupings to better understand what features make movies succeed. The analysis grouped 2,632 movies into four distinct clusters ($K=4$).

The results showed that these clusters commonly reflected real-world film categories, such as high-budget high-revenue blockbuster films, low-budget independent films, and films with strong audience ratings but with moderate to low revenue performance. Model evaluation showed strong stability, with cluster proportions and means across the training and testing sets being similar. The test-set silhouette width of 0.203 indicates reasonable separation among the real-world dataset. The nearest centroid approach also helps predict which cluster new movies would fall into.

The strength of this model is its ability to uncover structure without labeled data. However, a key weakness is its sensitivity to variable selection, scaling, and the need to predefine the number of clusters. Improvements could be made by adding features like film genre or nationality. Overall, K-means clustering proved to be an effective and useful tool to understand and summarize movie performance patterns.

Reference

<https://www.kaggle.com/datasets/tmdb/tmdb-movie-metadata>

Key R code

None

```
# Packages
library(dplyr)
library(tidyr)
library(ggplot2)
library(readr)
library(cluster)
library(factoextra)
library(GGally)
library(knitr)
library(scales)
library(pdist)

# Import data (TMDB movies + credits) and merge on movie id
movies_raw <- read_csv("archive/tmdb_5000_movies.csv", show_col_types = FALSE)
credits_raw <- read_csv("archive/tmdb_5000_credits.csv", show_col_types = FALSE)

credits_small <- credits_raw %>%
  select(movie_id, title, cast, crew) %>%
  rename(id = movie_id)

tmdb_raw <- movies_raw %>%
  inner_join(credits_small, by = "id", suffix = c("", "_credits"))

# Numeric clustering variables
num_vars <- c("budget", "revenue", "runtime",
             "popularity", "vote_average", "vote_count")

# Recode zero budget / revenue / runtime as missing (TMDB convention)
tmdb_raw <- tmdb_raw %>%
  mutate(
    budget = ifelse(budget == 0, NA, budget),
    revenue = ifelse(revenue == 0, NA, revenue),
    runtime = ifelse(runtime == 0, NA, runtime)
  )

# Cleaning: drop NAs, drop duplicate movie IDs
tmdb_step1 <- tmdb_raw %>%
  select(id, title, all_of(num_vars)) %>%
  drop_na()
tmdb_step2 <- tmdb_step1 %>% distinct(id, .keep_all = TRUE)

# Tukey IQR rule to remove outliers on the numeric predictors
iqr_outlier_flag <- function(x) {
  q1 <- quantile(x, 0.25, na.rm = TRUE)
  q3 <- quantile(x, 0.75, na.rm = TRUE)
  iqr <- q3 - q1
  x < (q1 - 1.5 * iqr) | x > (q3 + 1.5 * iqr)
}
outlier_matrix <- sapply(tmdb_step2[, num_vars], iqr_outlier_flag)
outlier_rows <- apply(outlier_matrix, 1, any)
tmdb_clean <- tmdb_step2[!outlier_rows, ]
```

```

# Visualizations
tmdb_long <- tmdb_clean %>%
  select(all_of(num_vars)) %>%
  pivot_longer(cols = everything(), names_to = "variable", values_to = "value")
ggplot(tmdb_long, aes(x = value)) +
  geom_histogram(bins = 30, fill = "steelblue", color = "white") +
  facet_wrap(~ variable, scales = "free") + theme_minimal()
ggplot(tmdb_long, aes(x = "", y = value)) +
  geom_boxplot(fill = "lightgreen") +
  facet_wrap(~ variable, scales = "free") + theme_minimal()

# Train / test split (70 / 30)
set.seed(123)
n <- nrow(tmdb_clean)
train_idx <- sample(seq_len(n), size = floor(0.70 * n))
train_data <- tmdb_clean[train_idx, ]
test_data <- tmdb_clean[-train_idx, ]
n_train <- nrow(train_data); n_test <- nrow(test_data)

# Standardize features using training-set mean and sd
train_x <- train_data %>% select(all_of(num_vars))
test_x <- test_data %>% select(all_of(num_vars))
train_means <- colMeans(train_x)
train_sds <- apply(train_x, 2, sd)
train_x_scaled <- scale(train_x, center = train_means, scale = train_sds)
test_x_scaled <- scale(test_x, center = train_means, scale = train_sds)

# Diagonal (pairs) plot to verify assumptions
pairs_sample <- as.data.frame(train_x_scaled) %>%
  slice_sample(n = min(500, nrow(train_x_scaled)))
ggpairs(pairs_sample, title = "Diagonal plot of standardized training features")

# Choose K: elbow, silhouette, gap statistic
set.seed(123)
fviz_nbclust(train_x_scaled, kmeans, method = "wss", k.max = 10)
fviz_nbclust(train_x_scaled, kmeans, method = "silhouette", k.max = 10)
gap_stat <- clusGap(train_x_scaled, FUN = kmeans, nstart = 25, K.max = 10, B = 50)
fviz_gap_stat(gap_stat)

# Fit final K-means model (K chosen from the plots above)
final_k <- 4
set.seed(123)
km_final <- kmeans(train_x_scaled, centers = final_k, nstart = 25, iter.max = 100)
km_final$size
km_final$tot.withinss
km_final$betweenss / km_final$totss

# Cluster plot on first two principal components
fviz_cluster(km_final, data = train_x_scaled,
  geom = "point", ellipse.type = "convex")

```

```

# Cluster centers transformed back to original (interpretable) scale
centers_orig <- sweep(km_final$centers, 2, train_sds, `*`)
centers_orig <- sweep(centers_orig, 2, train_means, `+`)
centers_orig_df <- as.data.frame(centers_orig) %>%
  mutate(cluster = factor(1:final_k), size = km_final$size) %>%
  relocate(cluster, size)
centers_orig_df

# Predict cluster membership for the test set (nearest training centroid)
predict_kmeans <- function(centers, newdata) {
  d <- as.matrix(pdist::pdist(X = as.matrix(newdata), Y = as.matrix(centers)))
  apply(d, 1, which.min)
}
test_clusters <- predict_kmeans(km_final$centers, test_x_scaled)
train_clusters <- km_final$cluster

# Model evaluation: cluster stability (train vs test proportions)
stability_df <- data.frame(
  cluster = factor(1:final_k),
  train_proportion = as.numeric(prop.table(table(train_clusters))),
  test_proportion = as.numeric(prop.table(table(test_clusters)))
)
stability_df

# Silhouette analysis on the test set
test_dist <- dist(test_x_scaled)
sil_test <- silhouette(test_clusters, test_dist)
summary(sil_test)[["avg.width"]]
fviz_silhouette(sil_test)

# Test-set cluster profile on the original scale
test_profile <- test_data %>%
  mutate(cluster = factor(test_clusters)) %>%
  group_by(cluster) %>%
  summarise(across(all_of(num_vars), mean), .groups = "drop") %>%
  mutate(size = as.integer(table(test_clusters)))
test_profile

# Example movies from each test-set cluster (prediction / interpretation)
example_movies <- test_data %>%
  mutate(cluster = factor(test_clusters)) %>%
  group_by(cluster) %>%
  slice_max(order_by = popularity, n = 5) %>%
  select(cluster, title, budget, revenue, runtime,
         popularity, vote_average, vote_count) %>%
  arrange(cluster)
example_movies

```